

Multilingual Topic Generation for Nigerian Languages **Competition Brief**

Track: LLM / Agent

Goal: Use a large language model to do two jobs on the same Nigerian-language news article — say what topic it is about and write a headline for it. Your final rank comes from a single merged score across both. [Click to access the assignment](#)

Format: Two submissions are required — predictions to the Kaggle leaderboard as a CSV, and the notebook that produced them, submitted for grading at the close of the competition. All four languages are scored together into one combined score.

Why these matters

- Nigeria has 500+ languages, and even the four biggest (Hausa, Igbo, Yoruba, Pidgin) are underserved in AI compared to English.
- Topic detection and headline generation power things you already use: news apps, content moderation, search, recommendations.
- The compute limits are deliberate. The point is to show that useful multilingual NLP does not require a cluster — a sub-1B model on a free GPU should get you there.

Dataset (already cleaned and ready)

Dataset: Derived from MasakhaNEWS (Adelani et al., 2023). Do not download it yourself. train.csv, dev.csv, and test.csv are provided on the competition's Kaggle page, already cleaned, shuffled, and with all four languages combined into single files.

Fields: train.csv and dev.csv contain five columns: id, language, text, headline, and label. test.csv contains three: id, language, and text. The headline and label values for the test rows are withheld and held on the leaderboard side.

Language	Train	Validation	Test
Hausa	2,219	317	637
Igbo	1,356	194	390
Nigerian Pidgin	1,060	152	305
Yoruba	1,433	206	411
All four (combined)	6,068	869	1,743

The Two Tasks

Every submission must do both tasks, and both are evaluated across all four languages at once. There is no single-language track: the test set is one combined file containing Hausa, Igbo, Yoruba, and Nigerian Pidgin rows, and your model must handle whatever it is given. A submission that does only one task can still be graded, but the missing half scores zero in the merge — so a weak attempt at both beats a strong attempt at one.

Task A — News Topic Classification

Input: A news article (text).

Output: Exactly one of 7 topic labels: business, health, politics, religion, sports, entertainment, technology.

Metric: Macro-F1, computed over the pooled test set (all four languages together, all 7 labels). Accuracy and per-language Macro-F1 are reported for information but do not enter the merged score — pooled macro-F1 keeps both the rare labels and the smaller languages honest.

Task B — News Headline Generation

Input: The same news article (text).

Output: A concise, relevant headline in the same language as the article. The language column tells you which language each row is in, so there is no language-detection step. **Metric:** A blended generation score, $\text{GenScore} = 0.5 \times \text{ROUGE-L(F1)} + 0.5 \times \text{BERTScore(F1)}$, computed over the pooled test set against the withheld headline column. A multilingual BERTScore backbone is used so all four languages are scored on the same footing. BLEU and human ratings for relevance, fluency, and factual accuracy may be reported in your notebook but are not scored.

Example:

Input: an article describing a new government programme providing financial support to small businesses.

Output: “Government Launches New Support Programme for Small Businesses”.

The Merged Score

Your leaderboard position is a single number combining both capabilities: Final

$$\text{Score} = 0.5 \times \text{Macro-F1 (Task A)} + 0.5 \times \text{GenScore (Task B)}$$

Both components are on a 0–1 scale, so neither task can dominate by unit alone. Equal weighting is deliberate: the competition rewards a model that can do both jobs, not one that is excellent at labelling and useless at writing.

Tie-break order, applied in sequence:

1. Higher pooled Macro-F1.
2. Smaller model parameter count.
3. Shorter total notebook runtime.

Rules and Compute Limits

These constraints are part of the competition, not advice. A submission that breaks them is not

scored.

1. Your scored model must be under 1B parameters. That is the point of the competition, not a preference — think mT5-small/base, AfriBERTa, XLM-R base, Qwen2.5-0.5B, SmolLM2-360M, or similar.
2. State your model's parameter count explicitly in the notebook. An unstated parameter count is treated as failing the size rule.
3. You may prompt a larger hosted model as a zero-shot reference point for comparison only. Every prediction in submission.csv must come from the sub-1B model you load and run inside the notebook.
4. Compute: Everything must run on a free Colab GPU or a Kaggle notebook — the budget is 1× T4, or 2× T4 on Kaggle. No A100s, no multi-node, no external training runs.
5. All training, fine-tuning, and inference happen in a single notebook. Do not upload checkpoints trained elsewhere as a private dataset. Graders reproduce your score by running your notebook from top to bottom.
6. The notebook must complete within the platform's session limit. Report your total runtime; it is the third tiebreaker.
7. LoRA or other parameter-efficient fine-tuning (via peft) is the expected route to fitting training inside this budget. Full fine-tuning of a sub-1B model is also fine if it fits.
8. All four languages are evaluated together. You do not pick a language — train one model that covers Hausa, Igbo, Yoruba, and Nigerian Pidgin, and predict on every row of test.csv.
9. Both tasks must be predicted for every row in test.csv. Missing rows are scored as wrong for Task A and empty for Task B.
10. train.csv and dev.csv are yours to use freely. Do not attempt to recover the withheld test labels or headlines from the original MasakhaNEWS release — doing so disqualifies the submission.
11. External unlabelled text for continued pre-training is allowed if it fits the compute budget and is cited in the notebook.

What You Need to Do

Steps 1–6 produce a complete, leaderboard-eligible entry. Do them in order. **Core steps (~4–5 hrs)**

1. Load train.csv, dev.csv, and test.csv from the competition page. Check the language distribution before anything else.
2. Zero-shot baseline, Task A: prompt a sub-1B instruct model to pick the topic label with no examples, across all four languages.
3. Zero-shot baseline, Task B: prompt the same model to write a headline for the same articles.
4. Improve the baselines: few-shot prompting, then LoRA fine-tuning of your sub-1B model. Fine-tuning is where the merged score usually jumps, particularly on Task B.
5. Score both tasks on dev.csv — pooled Macro-F1 and GenScore — and compute the merged score. Put baseline and final side by side in one table, with a per-language breakdown underneath.
6. Write 5–8 sentences: what moved the merged score, which task was harder, which language was hardest, and your guess why.

Good to know going in

- The four languages are unevenly sized — Hausa has roughly twice Pidgin's training rows. Expect your pooled score to be dragged by the smaller languages, and consider whether to rebalance.
- Headline generation is usually the harder half of the merged score. Expect Task B to be where the leaderboard separates.
- A sub-1B model will struggle with these languages out of the box. Reporting that honestly, with numbers, is part of the work.
- Watch your runtime. A submission that times out mid-notebook scores nothing, however good the method.

How to Submit

There are two parts, and both are required. Part 1 is automatic and scored by Kaggle; Part 2 is read by a human. A leaderboard score with no submitted notebook is disqualified, and a notebook that cannot reproduce the submitted CSV is disqualified — the leaderboard number only counts if the notebook backs it up.

Part 1 — Leaderboard submission (submission.csv)

Submissions are made directly on the Kaggle competition page — run your notebook, write submission.csv to the notebook output, then use Submit Predictions. Nothing is emailed or uploaded anywhere else. The public leaderboard scores a subset of the test rows during the competition; your final placement uses the private split, revealed at close. Standard Kaggle daily submission limits apply, so budget your attempts.

Submit a single CSV with two columns — id and prediction — in a long format. Each row of test.csv produces two submission rows: one for its topic label and one for its headline. The id column is the test row's id with a task suffix appended.

Example submission.csv:

```
id,prediction
```

```
hau_0001_topic,politics
```

```
hau_0001_headline,Gwamnati ta kaddamar da shirin tallafi ga kanana
```

```
sana'o'i_ibo_0002_topic,sports
```

```
ibo_0002_headline,Nna ndi egwuregwu Naijiria meriri asompi ahu
```

```
pcm_0003_topic,health
```

```
pcm_0003_headline,Government open new hospital for Lagos
```

Rules for the file: exactly 2 × the number of test rows, plus the header. Topic predictions must be one of the 7 labels in lowercase. Headline predictions are free text — quote fields containing commas, and never leave one blank. Missing or malformed ids are scored as wrong (Task A) or empty (Task B).

Part 2 — Notebook submission (required for grading)

The notebook must be submitted, not just linked. Make it public (or share it with the organisers) and post the notebook link on the competition page as instructed — this is what gets graded. •

It must run top to bottom on a single T4 (or 2× T4 on Kaggle) without manual intervention and write the exact submission.csv you submitted. Graders re-run it. • Inside the notebook: the merged-score table from your dev-set evaluation — Macro-F1, GenScore, and Final Score for your baseline and your best system, plus a per-language breakdown.

- A model card line: backbone name, parameter count, fine-tuning method, total runtime. 5–8 sentences of discussion, in a markdown cell.
- Anything else you want credited — error analysis, ablations, per-language notes — goes in this same notebook.

What counts as done (checklist)

- Notebook runs top to bottom without errors inside the GPU budget
- submission.csv written, 2 rows per test row, no blank predictions
- Notebook submitted and publicly viewable, and it regenerates that exact submission.csv
- All four languages, both tasks, predicted for every row of test.csv
- Model is under 1B parameters, and its parameter count is stated
- All training done in this one notebook — no external checkpoints
- Merged-score table present, with both components shown
- A few sentences on what you noticed